

BRICK BREAKER

FPGA Tabanlı Oyun Projesi

Platform: Nexys Video (xc7a200tsbg484-1)

Ekran: 128×32 OLED (SSD1306, SPI)

Dil: Verilog HDL

Araç: Vivado 2025.2

Saat: 100 MHz

Teknik Dokümantasyon Raporu

1. Genel Bakış

Brick Breaker, klasik tuğla kırma oyununun Nexys Video FPGA kartı üzerinde gerçekleşmesidir. Oyun, 128×32 piksel OLED ekranda çalışır ve tamamen donanım tabanlıdır — yazılım veya işlemci kullanılmamıştır. Temel Pong mekaniğine tuğla sistemi, sürpriz kutular, power-up'lar, can sistemi, duraklatma ve geri sayım özellikleri eklenmiştir.

Temel Özellikler

- 48 tuğla (3 satır × 16 sütun) kırma mekaniği
- 6 sürpriz tuğla: büyütme, küçültme, ekstra can
- Değişken paddle genişliği (10-30 piksel)
- Can sistemi (başlangıç: 1, maksimum: 3) — LED gösterimi
- BCD skor takibi (0-99)
- Duraklatma ve 3-2-1 geri sayım
- 6 durum makinesi: START, PLAY, PAUSE, COUNTDOWN, GAMEOVER, WIN
- Kombinasyonel frame buffer — RAM kullanılmaz

Giriş/Çıkış Tablosu

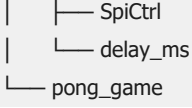
Sinyal	Yön	Pin	I/O Standardı	Açıklama
clk	Giriş	R4	LVC MOS33	100 MHz sistem saati
btnl	Giriş	C22	LVC MOS12	Sol hareket butonu
btnr	Giriş	D14	LVC MOS12	Sağ hareket butonu
btnc	Giriş	B22	LVC MOS12	Duraklatma butonu
btneu	Giriş	F15	LVC MOS12	Başlatma butonu
LED[7:0]	Çıkış	T14..Y13	LVC MOS25	Can göstergesi
oled_sdin	Çıkış	Y22	LVC MOS33	SPI MOSI
oled_sclk	Çıkış	W21	LVC MOS33	SPI Clock
oled_dc	Çıkış	W22	LVC MOS33	Data/Command
oled_res	Çıkış	U21	LVC MOS33	OLED Reset
oled_vbat	Çıkış	P20	LVC MOS33	VBAT kontrol
oled_vdd	Çıkış	V22	LVC MOS33	VDD kontrol

2. Sistem Mimarisi

Proje 6 Verilog modülünden oluşur. Üst modül (pong_top) butonları debounce eder, ~30 Hz oyun tick sinyali üretir ve alt modülleri bağlar. Frame buffer tamamen kombinasyoneldir: oled_driver adres üretir, pong_game o adrese karşılık gelen piksel verisini döndürür.

Modül Hiyerarşisi

```
pong_top (Üst Modül)
├── btn_debounce x4 (dbl, dbr, dbc, dbu)
└── oled_driver
```



Modül Açıklamaları

Modül	Dosya	İşlev
pong_top	pong_top.v	Üst modül: tick üretimi, debounce, bağlantı
btn_debounce	pong_top.v	20-bit sayaçlı buton debounce (~10 ms)
oled_driver	oled_driver.v	SSD1306 başlatma ve sürekli frame yenileme
SpiCtrl	SpiCtrl.v	8-bit SPI master kontrolcü
delay_ms	delay_ms.v	Milisaniye hassasiyetinde gecikme modülü
pong_game	pong_game.v	Tüm oyun mantığı ve piksel üretimi

Veri Akışı

- 1) pong_top 100 MHz saat ile çalışır, 22-bit sayaçla ~30 Hz game_tick üretir.
- 2) btn_debounce modülleri 4 butonu 2 kademeli senkronizasyon + ~10 ms debounce ile stabilize eder.
- 3) oled_driver sürekli olarak fb_addr[8:0] adreslerini tarar (4 sayfa × 128 sütun = 512 bayt).
- 4) pong_game kombinasyonel olarak fb_addr'e karşılık gelen fb_data[7:0] piksel baytını üretir.
- 5) oled_driver bu baytı SPI üzerinden OLED'e gönderir ve döngüyü tekrarlar.

3. Oyun Durumları (State Machine)

Oyun 3-bit gstate register'ı ile 6 durumda çalışır. Tüm durum geçişleri game_tick (30 Hz) kenarında gerçekleşir.

Durum	Değer	Açıklama	Çıkış Koşulu
ST_START	0	Başlık ekranı: PONG + PRESS UP	BTNU basılınca → COUNTDOWN
ST_PLAY	1	Aktif oyun	BTNC→PAUSE, tüm tuğla→WIN, can 0→OVER
ST_PAUSE	2	Oyun donmuş, PAUSE yazısı	BTNC → COUNTDOWN
ST_CD	3	3-2-1 geri sayım	Sayım bitince → PLAY
ST_OVER	4	GAME OVER + skor	~3 sn sonra → START
ST_WIN	5	YOU WIN + skor	~3 sn sonra → START

Durum Geçiş Detayları

START → COUNTDOWN: btn_u yükselen kenar algılanır (bu_edge). Tüm oyun değişkenleri sıfırlanır: top merkeze, paddle ortaya, tüm tuğlalar aktif, skor 0, can 1.

PLAY → PAUSE: btn_c yükselen kenar algılanır (bc_edge). Oyun anında durur.

PLAY → WIN: bricks register'ı 48'h000000000000 olduğunda (all_gone sinyali).

PLAY → GAMEOVER: Top ekranın altına düştüğünde (ny ≥ 31) ve lives == 0.

PAUSE → COUNTDOWN: Tekrar bc_edge. cd_val=3 ile geri sayım başlar.

COUNTDOWN → PLAY: cd_val 3'ten 1'e düştüğünde (her adım ~30 tick = ~1 saniye).

GAMEOVER / WIN → START: go_timer 90'a ulaştığında (~3 saniye).

Kenar Algılama Mekanizması

Buton edge detection game_tick hızında yapılır. bc_prev ve bu_prev register'ları her game_tick'te güncellenir:

bc_edge = btn_c & ~bc_prev

bu_edge = btn_u & ~bu_prev

Bu yaklaşım, debounce edilmiş seviye sinyalinin güvenli kenar çıkarır.

4. Tuğla Sistemi

48 tuğla tek bir 48-bit register'da (reg [47:0] bricks) saklanır. Başlangıçta tüm bitler 1'dir (48'hFFFFFFFFFFFF). Her tuğla kırıldığında ilgili bit 0 yapılır.

Fiziksel Düzen

OLED ekranın üst kısmında (sayfa 0, y=0-7) 3 satır tuğla bulunur:

- Satır 0: y = 0-1 (2 piksel yükseklik)
- Satır 1: y = 3-4 (2 piksel yükseklik)
- Satır 2: y = 6-7 (2 piksel yükseklik)

Her tuğla 8 piksel genişliğindedir. Sütun numarası col[6:3] ile belirlenir (0-15).

Adresleme

Tuğla indeksi 6 bit: {hrow[1:0], hcol[3:0]}. Topun y koordinatına göre satır belirlenir:

- $ny \leq 1 \rightarrow$ satır 0
- $ny \leq 4 \rightarrow$ satır 1
- $ny \leq 7 \rightarrow$ satır 2

Sütun: nx[6:3] ile hesaplanır.

Çarpışma Algılama

Top-tuğla çarpışması iki aşamalıdır:

- 1) Birincil kontrol: nx[6:3] sütunundaki tuğla canlı mı?
- 2) Sınır kontrolü: Eğer topun alt 3 biti 7 ise (nx[2:0]==3'd7) ve bir sonraki sütun da canlıysa, o tuğla kırılır.

Çarpışma gerçekleştiğinde:

- bricks[hidx] ← 0 (tuğla silinir)
- ball_dy ters çevrilir (dikey yön değişimi)
- Skor 1 artırılır (BCD)

Sayfa Tabanlı Rendering

Tuğlalar yalnızca sayfa 0'da (pg==0) render edilir. Her satır için bit maskesi:

- Satır 0: 8'b0000_0011 (bit 0-1)

- Satır 1: 8'b0001_1000 (bit 3-4)
- Satır 2: 8'b1100_0000 (bit 6-7)

Sürpriz tuğlalar çizgili desene sahiptir: yalnızca çift sütun piksellerinde render edilir (!col[0] koşulu).

5. Sürpriz Kutu ve Power-Up Sistemi

6 tuğla 'sürpriz' olarak işaretlenmiştir. Bu tuğlalar kırıldığında ekrandan düşen bir power-up ikonu belirir.

Sürpriz Tuğla Haritası

Tuğla İndeksi	Satır	Sütun	Tip	Etki
3	0	3	Büyütme (1)	Paddle +10 piksel
26	1	10	Büyütme (1)	Paddle +10 piksel
10	0	10	Küçültme (2)	Paddle = 10 piksel
35	2	3	Küçültme (2)	Paddle = 10 piksel
19	1	3	Ekstra Can (3)	Can +1 (maks 3)
44	2	12	Ekstra Can (3)	Can +1 (maks 3)

Düşen İkon Mekanizması

Power-up düşme mekanizması:

- 1) Sürpriz tuğla kırıldığında ve `pu_active==0` ise: `pu_active←1`, `pu_x` ve `pu_y` tuğlanın merkezine ayarlanır, `pu_type stype()` fonksiyonundan alınır.
- 2) Her `game_tick`'te `pu_tick` toggle edilir. Yalnızca `pu_tick==1` olduğunda ikon 1 piksel aşağı hareket eder (yarı hızda düşme).
- 3) İkon `y≥28`'e ulaştığında paddle çarpışma kontrolü yapılır:
 - Çarpışma: `pu_x+5 > paddle_x && pu_x < paddle_x+paddle_w`
 - Yakalanırsa: power-up uygulanır, `pu_active←0`
 - Yakalanamaz ve `y≥31`: `pu_active←0` (kaybolur)

İkon Görsel Farklılıkları

Her power-up tipi farklı genişlikte render edilir:

- Büyütme (tip 1): 5 piksel genişlik — geniş ikon
- Küçültme (tip 2): 1 piksel genişlik — ince ikon
- Can (tip 3): 3 piksel genişlik — orta ikon

İkon yüksekliği 3 piksel (`pumask10 = 10'h07 << pu_bit`). Sayfa sınırını aşarsa iki sayfada render edilir (`pumask0 + pumask1`).

6. Top Fiziği ve Paddle Etkisi

Top 2×2 piksel boyutundadır. Dikey hareket her tick'te 1 piksel, yatay hareket ise `ball_dx_skip` flag'ine bağlı olarak değişir.

Yatay Hız Kontrolü (Skip Mekanizması)

İki hız modu vardır:

- Normal (ball_dx_skip=0): Her tick'te 1 piksel yatay hareket. Topun 45° açıyla hareketi. Efektif hız: (1, 1) piksel/tick.
- Yavaş (ball_dx_skip=1): Her iki tick'te 1 piksel yatay hareket. ball_dx_cnt toggle register'ı ile sağlanır. Efektif hız: (0.5, 1) piksel/tick — daha dik açı.

Duvar çarpışması: $x==0$ 'da $ball_dx_dir \leftarrow 0$ (sağa), $x \geq 126$ 'da $ball_dx_dir \leftarrow 1$ (sola). Tavan çarpışması: $y==0$ 'da $ball_dy \leftarrow 0$ (aşağı).

Paddle Bölge Tespiti

Paddle çarpışmasında topun vuruş pozisyonu analiz edilir. Paddle 4 bölgeye ayrılır:

$quarter = paddle_w \gg 2$

- Sol çeyrek ($hpos < quarter$): $ball_dx_dir \leftarrow 1$ (sola), $ball_dx_skip \leftarrow 0$ (hızlı)
- Sağ çeyrek ($hpos \geq paddle_w - quarter$): $ball_dx_dir \leftarrow 0$ (sağa), $ball_dx_skip \leftarrow 0$
- Orta yarı: $ball_dx_skip \leftarrow 1$ (yavaş yatay). Yön paddle hareketine bağlı:
 - Paddle sola gidiyorsa (p_ml) $\rightarrow ball_dx_dir \leftarrow 1$
 - Paddle sağa gidiyorsa (p_mr) $\rightarrow ball_dx_dir \leftarrow 0$
 - Paddle duruyorsa $\rightarrow$ mevcut yön korunur

Paddle Çarpışma Koşulu

Çarpışma koşulu: $ny \geq 29 \ \&\& \ ball_dy == 0$ (top aşağı gidiyor)

$\&\& nx+2 > paddle_x \ \&\& \ nx < paddle_x + paddle_w$

Çarpışmada $ball_dy \leftarrow 1$ (yukarı) ve ny 28'e sabitlenir (sızma önlenir).

7. Can Sistemi ve Skor

Can Yönetimi

Başlangıç canı: 1 (lives register, 2-bit). Maksimum: 3.

Top kaybedildiğinde ($ny \geq 31$):

- $lives > 0$: Can 1 azalır, top ve paddle sıfırlanır, oyun devam eder
- $lives == 0$: Durum GAMEOVER'a geçer

Can kazanma: Tip 3 sürpriz tuğla yakalandığında $lives < 3$ ise +1.

LED Gösterimi

Can sayısı LED'lerle termometre kodlamasıyla gösterilir:

$led_out = \{5'b0, (lives \geq 3), (lives \geq 2), (lives \geq 1)\}$

- 1 can: LED[0] yanık
- 2 can: LED[1:0] yanık
- 3 can: LED[2:0] yanık

BCD Skor Sistemi

Skor iki BCD basamağından oluşur: sc_tens (onlar) ve sc_ones (birler).

Her tuğla kırıldığında artış:

```
if (sc_ones == 9) begin sc_ones ← 0; sc_tens ← sc_tens + 1; end
else sc_ones ← sc_ones + 1;
```

Skor GAMEOVER ve WIN ekranlarında sayfa 2'de, col 55-65'te gösterilir.

8. Duraklatma ve Geri Sayım

BTNC Kenar Algılama

BTNC (center button, pin B22) debounce sonrası seviye çıkışı verir. pong_game içinde game_tick hızında kenar algılama yapılır:

```
bc_prev her game_tick'te güncellenir
bc_edge = btn_c & ~bc_prev (yükselen kenar)
```

Bu yöntem, 30 Hz örnekleme ile güvenilir tek basım algılama sağlar.

PAUSE Durumu

PLAY durumunda bc_edge algılandığında:

- gstate ← ST_PAUSE
- Oyun mantığı çalışmaz (tüm hareket durur)
- Ekranda mevcut oyun sahnesinin üzerine 'PAUSE' yazısı OR ile eklenir
- PAUSE yazısı sayfa 1'de, col 47-75 arasında render edilir

COUNTDOWN Akışı

PAUSE'dan çıkışta veya START'tan başlarken COUNTDOWN devreye girer:

- 1) cd_val ← 3, cd_timer ← 0
- 2) Her tick'te cd_timer artırılır
- 3) cd_timer ≥ 29 olduğunda (~1 saniye):
 - cd_val == 1 ise → PLAY'e geç
 - Değilse cd_val ← cd_val - 1, cd_timer ← 0
- 4) Ekranda sayfa 1, col 61-65'te büyük rakam gösterilir

Geri sayım sırasında oyun sahnesi görünür (tuğlalar, top, paddle) ancak hareket etmez.

9. Piksel Üretimi (Kombinasyonel Frame Buffer)

Oyun RAM tabanlı frame buffer kullanmaz. oled_driver fb_addr[8:0] adresini üretir, pong_game tamamen kombinasyonel mantıkla fb_data[7:0] döndürür.

Adres Yapısı

$fb_addr[8:0] = \{sayfa[1:0], sütun[6:0]\}$

- Sayfa (pg): $fb_addr[8:7] \rightarrow 0-3$ (her sayfa 8 satır)
- Sütun (col): $fb_addr[6:0] \rightarrow 0-127$
- Toplam: $4 \times 128 = 512$ adres
- Her bayt 8 dikey piksel temsil eder (LSB = üst piksel)

Render Katmanları

Piksel baytı OR işlemiyle katman katman oluşturulur:

- 1) Tuğlalar ($pg==0$): 3 satır tuğla bit maskeleri
- 2) Top (herhangi sayfa): 2×2 piksel, $bmask0/bmask1$ ile sayfa sınırı aşar
- 3) Paddle ($pg==3$): bit 6-7, $paddle_x$ 'ten $paddle_w$ genişliğinde
- 4) Power-up (herhangi sayfa): pu_type 'a göre değişen genişlik
- 5) Metin overlay: Duruma bağlı (PAUSE, geri sayım rakamı)

GAMEOVER ve WIN durumlarında yalnızca metin ve skor render edilir.

Top Render Mekanizması

Top 2×2 pikseldir. Sayfa sınırını geçtiğinde iki sayfada render edilir:

$bbit = ball_y[2:0]$ (sayfa içi bit pozisyonu)
 $bpg = ball_y[4:3]$ (sayfa numarası)
 $bmask9 = 9'h03 \ll bbit$ (2-bit maske, 9 bit genişlikte shift)
 $bmask0 = bmask9[7:0]$ (mevcut sayfa maskesi)
 $bmask1 = \{7'b0, bmask9[8]\}$ (sonraki sayfa taşma maskesi)

Font Sistemi

İki font fonksiyonu kullanılır:

- $fnt(digit, col)$: 0-9 rakamları, 5×7 piksel, sütun-öncelikli
- $lfnt(char, col)$: A-Z harfleri (17 harf tanımlı), 5×7 piksel

Her karakter 5 sütun genişliğinde, aralarında 1 sütun boşluk bırakılır. Metin pozisyonları sabit kodlanmıştır ve always @(*) blokları ile belirlenir.

10. Zamanlama Analizi

Projedeki tüm zamanlama parametreleri aşağıda özetlenmiştir.

Parametre	Değer	Hesaplama
Sistem saati	100 MHz	10 ns periyot (XDC: create_clock -period 10.000)
Game tick	~ 30 Hz	$100 \text{ MHz} / 3,333,334 = 29.97 \text{ Hz}$
Debounce süresi	~ 10 ms	$2^{19} / 100 \text{ MHz} \approx 5.24 \text{ ms}$ (bit 19 kontrolü)
Paddle hızı	3 px/tick	~ 90 piksel/saniye

Top dikey hız	1 px/tick	~30 piksel/saniye
Top yatay (normal)	1 px/tick	~30 piksel/saniye
Top yatay (skip)	0.5 px/tick	~15 piksel/saniye
Power-up düşme	0.5 px/tick	~15 piksel/saniye (pu_tick toggle)
Geri sayım adımı	~1 saniye	30 tick (cd_timer 0-29)
GAMEOVER/WIN süresi	~3 saniye	90 tick (go_timer 0-89)
SPI saat	~10 MHz	100 MHz / 10 (COUNTER_MAX=9)
OLED frame rate	~37 fps	512 bayt × 10 SPI çevrim ≈ 51.2 µs/bayt
delay_ms çözünürlük	1 ms	100,000 saat çevrimi (MAX=99999)

11. Proje Dosya Yapısı

Dosya	Satır	İçerik
pong_top.v	~80	Üst modül + btn_debounce — tick üretimi, 4 buton debounce
pong_game.v	~320	Oyun mantığı: durum makinesi, tuğla, top, paddle, power-up, font, render
oled_driver.v	~80	SSD1306 OLED sürücü — başlatma sekansı ve sürekli frame yenileme
SpiCtrl.v	~50	8-bit SPI master — 4 durumlu FSM, ~10 MHz SCLK
delay_ms.v	~40	Milisaniye gecikme — 3 durumlu FSM, 100 MHz bazlı
nexys_video_pong.xdc	~25	Pin atamaları ve saat kısıtı (Nexys Video)
build_and_program.tcl	~35	Vivado sentez, implementasyon, bitstream, programlama

12. Sonuç

Brick Breaker projesi, temel Pong mekaniğinden başlayarak kapsamlı bir FPGA oyun tasarımına evrilmiştir. Projenin öne çıkan tasarım kararları:

- Kombinasyonel frame buffer: RAM kullanmadan, adres-tabanlı piksel üretimi ile kaynak tasarrufu sağlanmıştır. oled_driver adres üretir, pong_game anlık olarak piksel verisini döndürür.
- Tek register'da tuğla yönetimi: 48 tuğla tek bir 48-bit register'da tutularak hem depolama hem de çarpışma kontrolü basitleştirilmiştir.
- Skip mekanizması ile top fiziği: Sadece bir flag ile iki farklı yatay hız elde edilmiş, bölme veya çarpma devresi gerektirmeden eğimli top hareketi sağlanmıştır.
- Paddle bölge sistemi: Paddle'ın farklı bölgelerine çarpan topun farklı açılarda sekmesi, oyuna stratejik derinlik katmıştır.
- Sürpriz tuğla ve power-up mekanizması: Düşen ikon sistemi ile oyuna dinamizm eklenmiş, üç farklı etki (büyütme, küçültme, can) oyun dengesini zenginleştirmiştir.
- Durum makinesi tasarımı: 6 durumlu FSM ile başlatma, duraklatma, geri sayım ve bitiş ekranları düzgün yönetilmiştir.
- Kenar algılama stratejisi: Debounce çıkışından game_tick hızında kenar çıkarma, basit ve güvenilir bir yaklaşımdır.

Tüm tasarım tamamen donanım tabanlıdır — işlemci veya yazılım kullanılmamıştır. Proje, Vivado 2025.2 ile sentezlenmiş

ve Nexys Video kartı üzerinde başarıyla çalıştırılmıştır.